

# 创新实践课程设计

课 程 设 计 题 目：基于大语言模型的多 Agent 可视化编排平台

学 院 名 称：网络空间安全学院

专     业：网络空间安全

班     级：网安 242 班

姓     名：蔡宇涵

学     号：24415010226

定稿日期：2026 年 6 月

## 摘 要

随着大语言模型技术的快速发展，多 Agent 协同 workflow 在软件工程、数据分析、自动化运维等领域展现出广泛的应用前景。大语言模型在自然语言理解、代码生成、逻辑推理等方面的能力提升，使得构建能够自主决策和协作的智能 Agent 成为可能。然而，现有多 Agent 编排工具存在交互方式单一、依赖声明复杂、缺乏运行时监控等问题，难以满足非技术用户的使用需求。

本文设计并实现了一个名为 GrassFlow 的可视化多 Agent 积木编排平台。平台采用声明式领域特定语言(DSL)描述 Agent 之间的依赖关系，系统自动解析执行顺序和并行调度。用户只需通过简洁的文本语法声明谁依赖谁，系统即可自动完成拓扑排序、并行分组、异步调度等复杂的执行逻辑。这种声明式的编程范式大大降低了多 Agent 工作流的创建门槛。

平台的核心架构包含四个关键模块。第一是基于 Kahn 算法的 DAG 拓扑排序引擎，负责解析 Agent 依赖关系并生成并行执行分组，能够在构造阶段快速检测环形依赖并抛出异常。第二是基于 asyncio 的异步调度器，在组内通过 asyncio.gather 并发执行 Agent，在组间保持顺序约束，同时支持批处理和流式两种执行模式。第三是支持工具调用的 LLM Agent，通过迭代式工具调用循环(最多 10 轮)实现与文件读写、代码搜索、Shell 命令执行等外部工具的交互，并支持按 Agent 粒度配置工具权限。第四是基于规则的中文意图检测模块，能够识别先 X 再 Y 最后 Z、对比 X 和 Y 等自然语言模式，自动将其转换为 DSL 工作流定义。

系统采用 Python 语言开发，后端基于 FastAPI 框架，终端界面使用 Rich 和 prompt\_toolkit 库实现。项目包含 112 个 Python 源文件，共计约 58000 行代码，并通过 1372 个自动化测试用例验证了功能的正确性。测试结果表明，DAG 引擎能够正确处理包含扇出和扇入结构的复杂依赖图，调度器在并行执行场景下能够有效利用异步 IO 特性提升吞吐量，工具调用机制能够可靠地完成文件读写、代码搜索等操作。

**关键词：**多 Agent 编排；大语言模型；DAG 调度；领域特定语言；工具调用

## ABSTRACT

With the rapid development of large language model technology, multi-agent collaborative workflows have shown broad application prospects in software engineering, data analysis, and automated operations. The advancement of LLM capabilities in natural language understanding, code generation, and logical reasoning has made it possible to build intelligent agents that can make autonomous decisions and collaborate effectively. However, existing multi-agent orchestration tools suffer from monotonous interaction methods, complex dependency declarations, and lack of runtime monitoring, making them difficult for non-technical users to adopt.

This thesis designs and implements a visual multi-agent block orchestration platform called GrassFlow. The platform uses a declarative domain-specific language to describe dependencies between agents, with the system automatically resolving execution order and parallel scheduling. Users only need to declare who depends on whom through concise text syntax, and the system automatically completes topological sorting, parallel grouping, and asynchronous scheduling. This declarative programming paradigm significantly lowers the barrier to creating multi-agent workflows.

The core architecture consists of four key modules: a DAG topological sorting engine based on Kahn's algorithm for parsing agent dependencies and generating parallel execution groups, with fast cycle detection at construction time; an asyncio-based asynchronous scheduler that executes agents concurrently within groups using `asyncio.gather` while maintaining sequential constraints between groups, supporting both batch and stream execution modes; an LLM agent with tool calling capability that interacts with external tools through an iterative tool calling loop with a maximum of 10 rounds, supporting per-agent permission filtering; and a rule-based intent detection module that recognizes Chinese natural language patterns such as first X then Y and compare X with Y, automatically converting them into DSL workflow definitions.

The system is developed in Python with the backend based on the FastAPI framework and the terminal interface implemented using Rich and `prompt_toolkit` libraries. The project contains 112 Python source files with approximately 58,000 lines of code. System correctness has been verified through 1,372 automated test cases covering all major modules. Tests run in GitHub Actions CI environment across Python 3.10, 3.11, and 3.12. Results show that the DAG engine correctly handles complex dependency graphs with fan-out and fan-in structures, the scheduler effectively utilizes asynchronous

IO for throughput improvement, and the tool calling mechanism reliably completes file operations and code searching tasks.

**Key Words:** Multi-agent orchestration; Large language model; DAG scheduling; Domain-specific language; Tool calling

## 目 录

|                              |    |
|------------------------------|----|
| 摘 要 .....                    | 2  |
| 第 1 章 绪论.....                | 7  |
| 1.1 项目背景 .....               | 7  |
| 1.2 项目目标及意义 .....            | 7  |
| 1.3 本文结构 .....               | 8  |
| 第 2 章 需求分析.....              | 9  |
| 2.1 系统概述 .....               | 9  |
| 2.2 功能需求 .....               | 9  |
| 2.2.1 DSL  workflow 定义 ..... | 9  |
| 2.2.2 DAG 调度执行 .....         | 9  |
| 2.2.3 LLM Agent 与工具调用 .....  | 10 |
| 2.2.4 自然语言意图检测 .....         | 10 |
| 2.2.5 监控与可视化 .....           | 10 |
| 2.3 非功能需求 .....              | 10 |
| 第 3 章 设计方案.....              | 11 |
| 3.1 总体设计 .....               | 11 |
| 3.1.1 系统架构 .....             | 11 |
| 3.1.2 技术选型 .....             | 11 |
| 3.2 详细设计 .....               | 12 |
| 3.2.1 DSL v2 语法设计 .....      | 12 |
| 3.2.2 DAG 引擎设计 .....         | 12 |

|                                                                |    |
|----------------------------------------------------------------|----|
| .....                                                          | 13 |
| 3.2.3 调度器设计 .....                                              | 13 |
| 3.2.4 LLM Agent 与工具调用设计 .....                                  | 14 |
| .....                                                          | 14 |
| 3.2.5 组件注册与发现机制 .....                                          | 14 |
| 3.2.6 意图检测设计 .....                                             | 14 |
| 3.3 数据模型设计 .....                                               | 15 |
| 第 4 章 系统实现.....                                                | 16 |
| 4.1 开发环境 .....                                                 | 16 |
| 4.2 核心模块实现 .....                                               | 16 |
| 4.2.1 DAG 引擎实现 .....                                           | 16 |
| 4.2.2 调度器实现 .....                                              | 16 |
| 4.2.3 LLM Agent 实现 .....                                       | 17 |
| 4.2.4 DSL 解析器实现 .....                                          | 17 |
| 4.2.5 REPL 终端实现 .....                                          | 18 |
| <div data-bbox="327 1592 1390 1693" data-label="Image"> </div> | 18 |
| 4.3 系统测试 .....                                                 | 18 |
| 第 5 章 总结与展望.....                                               | 20 |
| 5.1 总结 .....                                                   | 20 |
| 5.2 展望 .....                                                   | 20 |

|            |    |
|------------|----|
| 参考文献 ..... | 22 |
|------------|----|

## 第 1 章 绪论

### 1.1 项目背景

近年来,大语言模型(Large Language Model, LLM)技术取得了突破性进展。以 GPT-4、Claude、DeepSeek 为代表的大语言模型在自然语言理解、代码生成、逻辑推理等方面展现出接近人类水平的能力。2023 年 OpenAI 发布的 GPT-4 模型在多项基准测试中超越了人类平均水平,Anthropic 公司推出的 Claude 系列模型在长文本理解和代码分析方面表现突出,国内 DeepSeek 团队开发的 DeepSeek-V3 模型在中文理解和多轮对话方面取得了显著进展。这一技术进步催生了大量基于 LLM 的 AI 应用,其中多 Agent 协同工作流程成为一个重要的研究和应用方向。

多 Agent 工作流程是指将一个复杂任务分解为多个子任务,由不同的 Agent 分别执行,并通过预定义的依赖关系协调各 Agent 之间的数据传递和执行顺序。这种模式在多个领域具有广泛的应用价值。在软件工程领域,代码审查流程通常包含代码读取、复杂度分析、安全扫描、风格检查、审查报告生成等多个步骤,这些步骤之间存在明确的先后依赖和并行关系。在数据分析领域,多维度报告生成需要同时从不同角度分析数据,然后将各维度的分析结果汇总为综合报告。在自动化运维领域,故障诊断链路需要依次执行日志收集、异常检测、根因分析、修复建议生成等步骤。

然而,现有多 Agent 编排工具存在以下问题。第一,交互方式单一。Langflow、n8n 等工具主要提供图形化界面,虽然直观但不适合程序化操作和 AI 自动生成;CrewAI 等框架仅提供代码级接口,要求用户具备编程能力才能定义 workflow。第二,依赖声明复杂。用户需要显式管理 Agent 之间的数据传递和执行顺序,对于包含扇出、扇入、条件分支等复杂结构的工作流,手动管理依赖关系容易出错。第三,缺乏运行时监控机制。用户无法实时了解各 Agent 的执行状态、数据流转情况和执行时间统计,难以及时发现和定位问题。第四,Agent 能力配置不灵活。多数工具将工具权限作为全局配置,难以针对不同任务场景调整单个 Agent 的工具权限和模型参数。

### 1.2 项目目标及意义

本文设计并实现了一个名为 GrassFlow 的可视化多 Agent 积木编排平台。平台的核心设计理念是用户只需声明谁依赖谁,系统自动处理执行顺序和并行调度。取名 GrassFlow(野草+流程),寓意像野草一样自由蔓延,Agent 编排自然生长。具体而言,本文的工作包括以下几个方面。

第一,设计了一套声明式领域特定语言(DSL)。DSL 语法支持顺序执行(A->B)、并行执行((A,B)->C)、条件分支(-> [urgent] human)、广播分发(A->(B,C))、流式触发

(mode: stream)等多种交互模式。用户可以通过简洁的文本语法描述复杂的 Agent 依赖关系，也可以让 AI 根据自然语言描述自动生成 DSL 工作流。

第二，实现了基于 DAG 的调度引擎。引擎采用 Kahn 算法进行拓扑排序，自动生成并行执行分组。拓扑排序和环检测的时间复杂度均为  $O(V+E)$ ，其中  $V$  为节点数、 $E$  为边数。调度器基于 `asyncio` 实现组内并发调度，在保证依赖约束的前提下最大化并行度，适合 LLM API 调用等 I/O 密集型操作。

第三，实现了支持工具调用的 LLM Agent。Agent 通过迭代式工具调用循环(最多 10 轮)在执行过程中自主决定是否调用外部工具，包括文件读写、代码搜索、Shell 命令执行等。每个 Agent 组件可以独立配置允许和禁止使用的工具列表，实现按 Agent 粒度的最小权限控制。

第四，实现了基于规则的中文意图检测模块。模块能够识别用户自然语言中的多步骤任务描述模式，如先 X 再 Y 最后 Z、对比 X 和 Y、分别处理 X 等，自动将其转换为 DSL 工作流定义，并在用户确认后执行。这降低了工作流创建的技术门槛。

第五，实现了 REPL 交互终端和实时监控面板。REPL 终端基于 `prompt_toolkit` 库构建，支持命令历史、Tab 补全、流式输出等交互功能，提供 20 余个斜杠命令管理工作流和系统配置。监控面板基于 `Rich` 库实现，支持嵌入式和全屏两种显示模式，能够实时展示 DAG 执行进度、Agent 状态和数据流转情况。

## 1.3 本文结构

本文共分为五章。第 1 章为绪论，介绍项目背景、研究现状和项目目标。第 2 章进行需求分析，从功能需求和非功能需求两个维度明确系统应具备的能力。第 3 章阐述系统的总体设计和详细设计方案，包括系统架构、DSL 语法设计、DAG 引擎设计、调度器设计、LLM Agent 设计、组件系统设计等。第 4 章展示系统实现过程，包括开发环境、核心模块的实现细节、以及系统测试结果。第 5 章对全文进行总结，分析系统的特点和不足，并展望未来工作方向。



## 第 2 章 需求分析

### 2.1 系统概述

GrassFlow 是一个声明式多 Agent 编排平台，目标用户包括需要编排复杂 AI 工作流的技术人员和希望通过自然语言描述任务的非技术用户。系统采用 Python 语言开发，提供命令行工具(CLI)和交互式终端(REPL)两种使用方式，支持通过 DSL 语法定义工作流并自动调度执行。系统的设计目标是让用户只需声明 Agent 之间的依赖关系，系统自动完成拓扑排序、并行分组、异步调度等执行逻辑。

与同类工具相比，GrassFlow 的差异化特点在于：支持 GUI+TUI 双模式交互，其中 TUI 模式的 DSL 语法可由 AI 自动生成；支持丰富的交互类型，包括顺序、并行、条件分支、广播分发、聚合等待、立即执行、流式触发等；提供按 Agent 粒度的工具权限控制；以及内置监控 Agent 机制，用 Agent 监控 Agent 的执行质量。

### 2.2 功能需求

#### 2.2.1 DSL 工作流定义

系统应提供一套声明式 DSL 语法，支持用户定义 Agent 组件和工作流。组件定义方面，用户应能定义可复用的 Agent 组件模板，包括系统提示词(system\_prompt)、输入输出端口声明(port)、模型配置(model)、MCP 工具配置(mcp)、工具权限声明(permission)、执行模式(mode: batch/stream)、上下文策略(context: shared/independent)、失败策略(on\_fail: stop/skip/retry)等属性。

工作流定义方面，用户应能在工作流中实例化组件并覆盖部分配置(如模型温度、最大 token 数)，使用箭头语法声明 Agent 之间的依赖关系。连接语法应支持端口级精确连接(A.port1 -> B.port2)和自动推断连接(A -> B，自动匹配第一个可用端口)。交互模式方面，DSL 应支持顺序执行(A -> B)、并行执行((A, B) -> C)、立即执行(A | B)、条件分支(-> [urgent] human, [normal] bot)、广播分发(A -> (B, C, D))等模式。

#### 2.2.2 DAG 调度执行

系统应能够解析 DSL 定义的工作流，构建有向无环图(DAG)。调度器应自动进行拓扑排序生成并行执行分组，在执行过程中严格遵循依赖约束。组内 Agent 应能并发执行，组间保持顺序。调度器应支持三种失败策略：停止(stop，终止整个工作流)、跳过(skip，用空结果继续)、重试(retry，重新执行指定次数)。失败策略应支持在 Agent 实例和组件两个层级配置，实例级配置优先于组件级配置。

### 2.2.3 LLM Agent 与工具调用

系统应支持通过大语言模型 API 执行 Agent 任务，兼容 OpenAI、Anthropic、DeepSeek 等多个模型提供商。Agent 在执行过程中应能够自主决定是否调用外部工具，工具调用结果应反馈给 LLM 进行后续处理。工具调用循环应有最大迭代次数限制，防止无限循环。每个 Agent 组件应能够独立配置允许(allow)和禁止(deny)使用的工具列表，实现最小权限原则。当 Agent 配置了权限列表时，系统应从全局工具注册表中创建过滤后的副本，Agent 只能使用被允许的工具。

### 2.2.4 自然语言意图检测

系统应能够识别用户自然语言中的多步骤任务描述模式。支持的模式包括：顺序模式(先 X 再 Y 最后 Z)、然后模式(X 然后 Y)、对比模式(对比 X 和 Y)、分别模式(分别处理 X、Y 和 Z)。检测到意图后，系统应自动生成对应的 DSL 工作流文本，以预览形式展示给用户，在用户确认后创建并执行工作流。

### 2.2.5 监控与可视化

系统应提供实时监控面板，展示工作流的 DAG 结构、各 Agent 的执行状态(等待/运行中/完成/失败)、数据流转情况和执行时间统计。监控面板应支持两种显示模式：嵌入式模式(在 REPL 终端内紧凑显示)和全屏模式(类似 htop 的全屏界面)。调度器应提供事件回调机制，支持工作流级别、分组级别、Agent 级别共 10 种事件类型的监听。

## 2.3 非功能需求

在性能方面，调度器应充分利用 asyncio 异步 IO 特性，在 Agent 执行 I/O 密集型任务(如调用 LLM API)时实现组内并发，减少总体执行时间。工具调用循环的单轮延迟应控制在合理范围内，避免因工具执行过慢导致整个工作流阻塞。

在可扩展性方面，组件系统应支持多级发现机制(文件内联、本地、项目、全局、编程方式)，允许在不同范围定义和复用组件。工具系统应提供统一的注册接口，支持内置工具和外部工具的扩展。MCP 协议集成应支持连接外部 MCP 服务器获取更多工具能力。

在可靠性方面，系统应对 DAG 进行环检测，确保工作流不存在循环依赖。Agent 输入输出应进行 Schema 校验，确保数据类型符合端口声明。工具调用应进行权限控制，防止 Agent 越权操作。系统应提供完善的错误处理和日志记录机制。

在可测试性方面，系统应具备完善的自动化测试覆盖。测试应覆盖核心引擎、调度器、DSL 解析器、工具系统、REPL 终端等所有主要模块。测试应在持续集成环境中自动运行，支持多 Python 版本验证。

## 第 3 章 设计方案

### 3.1 总体设计

#### 3.1.1 系统架构

GrassFlow 采用分层架构设计，自底向上分为核心引擎层、运行时层和用户界面层三个层次。各层之间通过明确定义的接口进行通信，层内模块之间保持低耦合、高内聚的设计原则。

核心引擎层包含 DAG 引擎、数据模型和组件注册表三个模块。DAG 引擎负责将工作流的连接关系转化为有向无环图，执行拓扑排序和环检测。数据模型定义了 Component、AgentInstance、Connection、Workflow 等核心类型，是 DSL 解析和运行时执行之间的桥梁。组件注册表负责组件的发现、注册和查询，支持 5 级优先级的就近发现机制。

运行时层包含调度器、LLM Agent 和工具系统三个模块。调度器按照 DAG 引擎生成的并行分组执行工作流，支持批处理和流式两种执行模式。LLM Agent 封装了与大语言模型的交互逻辑，支持迭代式工具调用。工具系统提供工具注册表和权限过滤机制，管理内置工具和外部工具的生命周期。

用户界面层包含 REPL 终端、CLI 命令行和监控面板三个模块。REPL 终端提供交互式对话和斜杠命令，CLI 命令行提供工作流的执行、验证、列表等操作，监控面板提供 DAG 可视化和执行状态展示。

#### 3.1.2 技术选型

系统的技术选型如下。后端语言采用 Python 3.9 及以上版本，利用其丰富的异步编程支持(asyncio)和 AI 生态库。Python 的 dataclass 装饰器用于定义数据模型，type hints 用于提高代码可读性。

Web 框架采用 FastAPI 0.100.0，用于提供 RESTful API 和 WebSocket 实时通信。FastAPI 基于 Pydantic 进行数据校验，支持异步请求处理，自动生成 OpenAPI 文档。ASGI 服务器采用 uvicorn，提供高性能的 HTTP 服务。

终端界面采用 Rich 13.0 库实现富文本渲染，包括表格、面板、进度条、语法高亮等。交互式输入采用 prompt\_toolkit 3.0 库，支持命令历史、Tab 补全、多行编辑等交互功能。

LLM 调用采用 LiteLLM 1.0 库，统一了 OpenAI、Anthropic、DeepSeek、Ollama 等多个模型提供商的接口。LiteLLM 处理了不同提供商之间的 API 差异、认证方式、流式响应格式等兼容性问题。

数据存储采用 SQLite，通过 aiosqlite 库实现异步访问。SQLite 用于保存执行记录、会话历史和工作流配置。配置管理采用 JSON 格式，支持全局(~/.Grass/)和项目(.grass/)两级配置。

## 3.2 详细设计

### 3.2.1 DSL v2 语法设计

DSL v2 语法包含两个顶层结构：组件定义(component)和工作流定义(workflow)。组件是可复用的 Agent 模板，包含系统提示词、端口声明、模型配置、MCP 工具配置和权限声明。工作流由 Agent 实例和连接关系组成，Agent 可以引用已定义的组件并覆盖部分配置。

组件定义的语法结构如下。component 关键字后跟组件名称和大括号包围的属性块。属性包括：description(组件描述)、system\_prompt(系统提示词，支持多行文本)、port input/output(端口声明，包含名称、类型和描述)、model default/fallback/temperature/max\_tokens(模型配置)、mcp server\_name { tools: [...] }(MCP 工具配置)、permission allow/deny/ask(工具权限声明)。

工作流定义的语法结构如下。workflow 关键字后跟工作流名称和大括号包围的主体。主体中包含 Agent 声明和连接关系。Agent 声明支持两种形式：内联定义(agent name { model: ..., prompt: ... })和组件引用(agent name use component\_name { model temperature: 0.5 })。连接关系使用->符号声明，支持端口级精确连接(A.port -> B.port)和自动推断(A -> B)。

连接语法支持五种模式。顺序执行(A -> B)表示 A 完成后执行 B。并行执行((A, B) -> C)表示 A 和 B 同时执行，都完成后执行 C。立即执行(A | B)表示 A 和 B 同时启动，遇到依赖时等待。条件分支(-> [urgent] human)表示根据上游 Agent 输出的 route 字段选择执行路径。流式触发(mode: stream)表示上游输出列表时逐项触发下游 Agent。

### 3.2.2 DAG 引擎设计

DAG 引擎负责将工作流的连接关系转化为有向无环图，并进行拓扑排序和并行分组。引擎在构造时即执行环检测，采用基于 DFS 的算法。算法维护一个访问集合(visited)和一个递归栈(recursion\_stack)，对每个未访问节点执行深度优先搜索。当发现某个节点的邻居在当前递归路径中重复出现时，判定存在环，立即抛出 DAGError 异常。这种快速失败的设计确保调度器不会接收到无效的图结构。

拓扑排序采用 Kahn 算法(BFS 方式)。算法首先计算所有节点的入度，将入度为零的节点加入队列。然后迭代地从队列中取出节点，将其后继节点的入度减一，当后继节点入度变为零时加入队列。如果最终排序结果的节点数不等于图中总节点

数，说明存在环，抛出 `DAGError` 异常。算法的时间复杂度为  $O(V+E)$ ，其中  $V$  为节点数、 $E$  为边数。

并行分组在 Kahn 算法的基础上扩展。每次迭代时，将当前队列中的所有节点作为一个并行组捕获。同一组内的所有节点的依赖已全部满足，可以并发执行。分组数量等于 DAG 的最长路径长度(关键路径)。例如，对于包含扇出和扇入的菱形结构，分组结果为 4 组：根节点一组、两个扇出节点一组、扇入节点一组、叶子节点一组。

### 3.2.3 调度器设计

调度器按照 DAG 引擎生成的并行分组依次执行。组间采用顺序执行策略，确保依赖约束不被违反。组内采用 `asyncio.gather` 实现并发执行，多个 Agent 作为 `asyncio.Task` 同时启动，利用事件循环在 I/O 等待期间切换任务。这种设计适合 LLM API 调用等 I/O 密集型操作，能够在等待一个 Agent 的 API 响应时执行其他 Agent 的工具调用。

调度器支持两种执行模式。批处理模式(batch)是默认模式，Agent 处理单个输入并返回单个输出。流式模式(stream)是扩展模式，调度器从上游 Agent 的输出中提取列表数据，对列表中的每个元素分别触发下游 Agent 执行。context 属性控制上下文策略：shared 模式复用同一个 Agent 实例，对话历史在多次调用间累积，适用于需要保持上下文的场景；independent 模式为每次调用创建新的 Agent 实例副本，各次调用相互独立，适用于无状态的批处理场景。

调度器支持三种失败处理策略。stop 策略在任何 Agent 失败时立即终止整个工作流，适用于关键路径上的 Agent。skip 策略跳过失败的 Agent，用空结果继续执行后续 Agent，适用于可选的辅助 Agent。retry 策略在失败后重新执行，配合 `retry_count` 参数控制最大重试次数，适用于因网络波动等原因可能临时失败的 Agent。策略优先级为 Agent 实例覆盖高于组件默认值。

调度器提供事件回调机制，定义了 10 种事件类型。工作流级别包括 `WORKFLOW_START`、`WORKFLOW_COMPLETE`、`WORKFLOW_FAILED`。分组级别包括 `GROUP_START`、`GROUP_COMPLETE`。Agent 级别包括 `AGENT_START`、`AGENT_COMPLETE`、`AGENT_FAIL`、`AGENT_RETRY`、`AGENT_SKIPPED`。事件数据包含事件类型、Agent 名称、时间戳和附加数据。当未注册回调时，事件发射函数 `_emit` 直接返回，不产生任何开销。回调函数中的异常被静默捕获，确保不会中断工作流执行。

### 3.2.4 LLM Agent 与工具调用设计

LLM Agent 的执行流程包含一个迭代式工具调用循环。Agent 首先将系统提示词中的变量占位符(`{input}`、`{task}`、`{dep_*}`)替换为实际输入值，然后与用户输入一起构建消息列表，发送给 LLM。如果 LLM 的响应包含 `tool_calls` 字段，Agent 进入工具调用流程：解析每个工具调用的 `function.name` 和 `function.arguments`，通过 `ToolRegistry` 的 `invoke` 方法执行对应的工具，将执行结果构建为 `tool` 角色消息追加到对话中，然后再次发送给 LLM。这个过程最多重复 10 次，直到 LLM 返回纯文本响应或达到最大迭代次数。

工具权限过滤采用创建过滤注册表的机制。每个组件可以声明允许(`allow`)和禁止(`deny`)的工具 ID 列表。在 Agent 实例化时，系统调用 `create_filtered_registry` 函数，根据权限声明从全局工具注册表中创建一个过滤后的副本。如果 `allow` 列表非空，只保留 ID 在列表中的工具；如果 `deny` 列表非空，移除 ID 在列表中的工具；两者同时存在时先过滤 `allow` 再过滤 `deny`。Agent 只能使用过滤后注册表中的工具，实现了按 Agent 粒度的最小权限控制。

### 3.2.5 组件注册与发现机制

组件注册表采用就近优先的 5 级发现机制。第一级为文件内联(`FILE_INLINE`，优先级 0)，即在当前 `.gf` 文件中定义的组件。第二级为本地(`LOCAL`，优先级 1)，即 `./grass/components/` 目录。第三级为项目级(`PROJECT`，优先级 2)，即项目根目录下的 `.grass/components/`。第四级为全局(`GLOBAL`，优先级 3)，即用户主目录下的 `~/.Grass/components/`。第五级为编程方式(`PROGRAMMATIC`，优先级 4)，即通过代码注册的组件。当同名组件出现在多个级别时，高优先级的源覆盖低优先级的源，同时输出警告信息。

组件注册表提供丰富的查询接口。`get(name)` 方法根据名称精确查询组件。`search(query)` 方法在组件名称、描述、端口名称、MCP 服务器名称中进行模糊搜索。`filter_by_source()`、`filter_by_model()`、`filter_by_mcp()` 方法支持按来源、模型、MCP 服务器过滤组件。`export_component()` 方法将组件序列化为 DSL 文本，支持导入导出。

### 3.2.6 意图检测设计

意图检测模块采用基于规则的模式匹配方法，专门针对中文自然语言设计。模块定义了 4 种检测模式，按优先级从高到低排列。第一种是多步骤顺序模式(优先

级最高), 匹配先 X 再 Y 最后 Z 句式。正则表达式提取 X、Y、Z 作为子任务, 生成执行模式为 `sequential` 的意图。第二种是然后模式, 匹配 X 然后 Y 句式, 提取两个子任务。第三种是对比模式, 匹配对比 X 和 Y 句式, 生成 `parallel_aggregate` 模式: 先分别分析 X 和 Y, 再汇总对比结果。第四种是分别模式, 匹配分别 X、Y 和 Z 句式, 按顿号、逗号或和字分割子任务, 生成 `parallel` 模式。

检测结果包含 `task_description`(任务描述)、`sub_tasks`(子任务列表)、`estimated_agents`(预估 Agent 数量)、`pattern`(执行模式)。`generate_dsl` 方法根据执行模式生成 DSL 工作流文本: `sequential` 模式生成链式连接, `parallel` 模式生成无连接的独立 Agent, `parallel_aggregate` 模式生成并行 Agent 加聚合 Agent 的扇入结构。

### 3.3 数据模型设计

系统的数据模型采用 Python `dataclass` 定义, 分为 AST 层和运行时层。AST 层定义在 `models.py` 中, 包含以下核心类型。`Port` 类型表示端口, 包含 `name`(名称)、`direction`(方向: `input/output`)、`type`(类型: `string/number/boolean/object/array`)、`description`(描述)等字段。`Component` 类型表示组件模板, 包含 `name`、`system_prompt`、`ports`、`mcp`、`model`、`permission`、`mode`(`batch/stream`)、`context`(`shared/independent`)、`on_fail`(`stop/skip/retry`)、`retry_count` 等字段。`AgentInstance` 类型表示工作流中的 Agent 实例, 包含 `name`、`component`(组件引用)、`overrides`(覆盖配置)等字段。`Connection` 类型表示连接关系, 包含 `source_agent`、`source_port`、`target_agents`、`target_ports`、`routing_rules` 等字段。`Workflow` 类型表示完整工作流, 包含 `name`、`ports`、`agents`、`connections`、`output_mappings` 等字段。

运行时层定义在 `execution.py` 中, 包含 `ExecutionRecord`(执行记录)和 `AgentExecution`(Agent 执行详情)等类型。`ExecutionRecord` 记录每次工作流执行的 `workflow_name`、`start_time`、`end_time`、`status`、`total_agents`、`completed_agents`、`failed_agents` 等信息。`AgentExecution` 记录每个 Agent 的 `input_data`、`output_data`、`start_time`、`end_time`、`status`、`error_message` 等详细信息。执行记录采用 SQLite 数据库持久化存储, 通过 `aiosqlite` 库实现异步访问。数据库包含 `executions` 表和 `agent_executions` 表, 分别存储工作流级别和 Agent 级别的执行记录。

## 第 4 章 系统实现

### 4.1 开发环境

系统的开发和运行环境如下。操作系统为 Windows 11 Professional，编程语言为 Python 3.12.4，包管理工具为 pip，虚拟环境为 venv。Web 框架为 FastAPI 0.100.0，ASGI 服务器为 uvicorn 0.23.0。终端界面库为 Rich 13.0 和 prompt\_toolkit 3.0。LLM 调用库为 LiteLLM 1.0，支持 OpenAI、Anthropic、DeepSeek 等多个模型提供商。数据库为 SQLite，通过 aiosqlite 0.19.0 异步访问。测试框架为 pytest 7.0 配合 pytest-asyncio 0.21。代码格式化工具为 black，静态检查工具为 ruff。项目代码通过 Git 进行版本控制，托管在 GitHub 平台(<https://github.com/ycqaq233/GrassFlow>)，使用 GitHub Actions 实现持续集成，每次推送自动在 Python 3.10、3.11、3.12 三个版本上运行测试。容器化部署使用 Docker 和 Docker Compose。

### 4.2 核心模块实现

#### 4.2.1 DAG 引擎实现

DAG 引擎的核心实现在 `core/dag.py` 中，共 234 行代码。DAG 类的构造函数接收一个 Workflow 对象，遍历其 `connections` 列表构建前向邻接表(`adj`)和反向邻接表(`reverse_adj`)。构造完成后立即调用 `_has_cycle` 方法执行环检测。

环检测方法 `_has_cycle` 实现 DFS 算法。方法维护 `visited` 集合和 `recursion_stack` 集合，对每个未访问节点调用递归辅助函数 `_dfs`。辅助函数将当前节点同时加入 `visited` 和 `recursion_stack`，遍历其所有邻居节点：如果邻居在 `recursion_stack` 中，说明存在环，返回 `True`；如果邻居未访问，递归调用 `_dfs`。递归返回后将当前节点从 `recursion_stack` 中移除。整个算法的时间复杂度为  $O(V+E)$ 。

拓扑排序方法 `topological_sort` 实现 Kahn 算法。方法首先计算所有节点的入度字典，将入度为零的节点加入 `deque`。然后进入循环：从 `deque` 左侧取出节点(保证 BFS 顺序)，加入排序结果列表，遍历其后继节点并将入度减一，当后继节点入度变为零时加入 `deque`。循环结束后检查排序结果长度是否等于节点总数，不相等则抛出 `DAGError`。

并行分组方法 `get_parallel_groups` 在 Kahn 算法基础上扩展。方法维护一个 `current_level` 列表，每次循环开始时将当前 `deque` 中所有元素快照到 `current_level` 中作为一组，然后正常执行 Kahn 算法的去边操作。返回的 `groups` 列表中每个元素是一个并行组，组内节点可并发执行。

#### 4.2.2 调度器实现

调度器的核心实现在 `core/scheduler.py` 中，共 498 行代码。Scheduler 类的构造函数接收 Workflow 对象、Agent 字典和可选的事件回调函数。`run` 方法是执行入口，



方法首先调用 DAG 引擎获取并行分组，然后依次遍历每个分组调用 `_execute_group` 方法。执行前后分别发射 `WORKFLOW_START` 和 `WORKFLOW_COMPLETE` 事件。

`_execute_group` 方法将组内 Agent 分为批处理(batch)和流式(stream)两类。批处理 Agent 通过 `asyncio.create_task` 创建异步任务，然后用 `asyncio.gather(*tasks, return_exceptions=True)` 并发等待所有任务完成。`gather` 的 `return_exceptions` 参数确保单个 Agent 的异常不会导致其他 Agent 被取消。流式 Agent 通过 `_execute_stream_agent` 方法逐项执行。

流式执行方法 `_execute_stream_agent` 首先调用 `_get_stream_items` 从上游 Agent 输出中提取第一个列表类型的值。然后遍历列表中的每个元素，根据 context 策略选择执行方式：`shared` 模式复用同一个 Agent 实例，将当前元素作为输入调用 `run` 方法，对话历史在多次调用间累积；`independent` 模式为每个元素 `deepcopy` 一个新的 Agent 实例副本，各次调用相互独立。单个元素的失败不中断后续元素的执行。

条件路由通过 `_should_execute` 方法实现。方法检查目标 Agent 的所有入边 (Connection 对象)，如果存在 `routing_rules` 字典，则从源 Agent 的输出中提取 `route` 字段值，检查是否在 `routing_rules` 中有对应的映射。如果 `route` 值匹配了某个规则，只有规则指定的目标 Agent 才会被执行。

### 4.2.3 LLM Agent 实现

LLM Agent 的实现在 `core/llm_agent.py` 中，共 390 行代码。LLMAgent 类继承自 Agent 基类，构造函数接收 Component 对象和可选的 ToolRegistry 对象。`run` 方法是执行入口，方法首先通过 `_format_prompt` 方法将系统提示词中的变量占位符替换为实际输入值，然后构建包含 `system` 和 `user` 角色的消息列表。

如果 Agent 配置了 ToolRegistry，`run` 方法调用 `_run_with_tools` 进入工具调用循环。循环最多执行 10 轮，每轮中方法将消息列表和工具 Schema 列表发送给 LLM。工具 Schema 通过 ToolRegistry 的 `to_llm_tool_list` 方法生成 OpenAI 兼容的 `function calling` 格式。如果 LLM 响应包含 `tool_calls` 字段，方法解析每个工具调用的 `function.name` 和 `function.arguments`，通过 ToolRegistry 的 `invoke` 方法执行工具，将执行结果构建为 tool 角色消息(包含 `tool_call_id`)追加到对话中，然后进入下一轮。当 LLM 返回纯文本响应时，循环结束。方法累计所有轮次的 token 使用量，返回包含结果文本、元数据和工具调用日志的 LLMResponse 对象。

### 4.2.4 DSL 解析器实现

DSL v2 解析器的实现在 `tui/dsl_parser_v2.py` 中，共 480 行代码。解析器采用纯正则表达式实现，不依赖外部解析器生成器(如 ANTLR、PLY)。这种设计选择简化了依赖关系，同时对于 DSL 的有限语法复杂度已经足够。

解析过程分为三个阶段。预处理阶段(`_preprocess` 方法)去除注释(以#开头的行)和多余空白, 将多行文本规范化为统一格式。组件解析阶段(`_parse_components` 方法)通过正则匹配 `component` 关键字, 使用 `_extract_block` 方法通过大括号深度计数提取块内容(处理嵌套大括号), 然后用独立的正则表达式解析 `system_prompt`、`port`、`model`、`mcp`、`permission` 等字段。

工作流解析阶段(`_parse_workflows` 方法)采用类似的方法提取 `agent` 声明和连接关系。解析器首先收集所有 `agent` 声明(内联定义和 `use` 引用)及其在文本中的位置, 按位置排序以保留声明顺序。然后从工作流主体中移除 `agent` 块, 对剩余文本逐行解析连接关系。连接解析方法 `_parse_connection_line` 按 `->` 符号分割左右两侧, 左侧处理单个 Agent 引用(A 或 A.port)和聚合语法((A, B, C)), 右侧处理单个 Agent 引用和广播语法((B, C, D))。解析结果构建为 `Connection` 对象列表。

### 4.2.5 REPL 终端实现

REPL 终端的实现在 `tui/repl.py` 中, 共 1589 行代码, 是系统中最大的单个模块。REPL 基于 `prompt_toolkit` 库构建 `Application`, 使用 `patch_stdout` 模式确保 Agent 的异步输出不会干扰用户输入。终端维护一个 `FileHistory` 实例实现命令历史持久化, 维护一个 `SlashCommandCompleter` 实例实现斜杠命令的 Tab 补全。

REPL 的核心调度逻辑在 `_process_user_input` 方法中。方法首先判断输入是否以 `/` 开头, 如果是则调用 `_handle_slash_command` 路由到对应的命令处理器; 否则调用 `_check_workflow_detection` 进行意图检测。如果检测到多步骤意图, 方法生成 DSL 工作流文本并调用 `_handle_workflow_confirmation` 让用户确认; 如果未检测到意图, 方法调用 `_dispatch_agent` 将消息发送给 Agent 处理。

REPL 集成了 20 余个斜杠命令, 通过 `slash_commands.py` 中的命令注册表管理。常用命令包括: `/help`(显示帮助信息)、`/run`(执行工作流)、`/generate`(生成工作流)、`/list`(列出已保存的工作流)、`/save`(保存工作流)、`/validate`(验证工作流语法)、`/config`(管理配置)、`/clear`(清除对话历史)、`/compact`(压缩上下文)、`/monitor`(显示监控面板)等。命令注册表支持 Tab 补全和别名。



```

n      显示帮助信息
history 查看执行历史
hello-skill Load skill: A simple example skill that demonstrates the SKILL.md format
    
```

## 4.3 系统测试

系统采用 `pytest` 框架进行自动化测试, 共包含 1372 个测试用例, 覆盖核心引擎、调度器、DSL 解析器、工具系统、REPL 终端等所有主要模块。测试在 GitHub

Actions 持续集成环境中运行，每次代码推送时自动在 Python 3.10、3.11、3.12 三个版本上执行。

DAG 引擎的测试(test\_dag.py)覆盖了拓扑排序、环检测、并行分组等核心算法。测试用例包含简单的线性依赖(A->B->C)、包含扇出和扇入的菱形结构(A->(B,C)->D)、包含多层嵌套的复杂依赖图、以及存在环的非法图。测试验证了排序结果的正确性、分组数量的合理性、以及环检测的异常抛出。

调度器的测试(test\_scheduler.py)覆盖了批处理执行、流式执行、条件路由、失败策略、事件回调等功能。测试使用 Mock Agent 模拟不同执行场景，验证了组内并发的正确性、依赖约束的严格性、失败策略的行为一致性、以及事件回调的触发时机。流式模式的测试验证了 shared 和 independent 两种上下文策略的差异。

DSL 解析器的测试(test\_dsl\_parser\_v2.py)覆盖了组件定义、工作流定义、连接语法、错误处理等。测试用例包含各种合法和非法的 DSL 文本，验证了解析结果的正确性和错误信息的可读性。工具系统的测试覆盖了工具注册、权限过滤、内置工具执行等功能。REPL 的测试覆盖了斜杠命令的注册、参数解析、Tab 补全等交互功能。

## 第 5 章 总结与展望

### 5.1 总结

本文设计并实现了一个名为 GrassFlow 的可视化多 Agent 积木编排平台。平台采用声明式 DSL 语法描述 Agent 依赖关系，通过 DAG 引擎自动解析执行顺序和并行调度，支持顺序执行、并行执行、条件分支、广播分发、流式触发等多种交互模式。用户只需声明谁依赖谁，系统自动完成拓扑排序、并行分组、异步调度等执行逻辑。

在技术实现方面，系统基于 Kahn 算法实现拓扑排序和并行分组，时间复杂度为  $O(V+E)$ 。调度器基于 asyncio 实现组内并发调度，适合 LLM API 调用等 I/O 密集型操作。LLM Agent 通过迭代式工具调用循环(最多 10 轮)实现与外部工具的交互，并支持按 Agent 粒度的工具权限控制。意图检测模块采用基于规则的中文模式匹配，能够自动将自然语言转换为 DSL 工作流。组件系统采用 5 级就近优先发现机制，支持从文件内联到全局的多级组件复用。

在工程实践方面，项目包含 112 个 Python 源文件，共计约 58000 行代码。系统通过 1372 个自动化测试用例验证了功能的正确性，测试覆盖 DAG 引擎、调度器、DSL 解析器、工具系统、REPL 终端等所有主要模块。系统提供了 Docker 容器化部署方案和 GitHub Actions 持续集成配置，支持多 Python 版本的自动化测试。

### 5.2 展望

未来工作可以从以下几个方向展开。第一，实现 MCP(Model Context Protocol) 协议集成，使 Agent 能够连接外部 MCP 服务器获取更多工具能力。当前系统已预留 MCP 配置接口，但实际连接和工具调用尚未实现。

第二，开发 Electron 桌面应用，提供基于 React Flow 的可视化拖拽界面，实现所见即所得的工作流编辑体验。GUI 模式将支持积木拖拽、多种连线类型(实线/虚线/粗线)、实时状态动画等交互功能。

第三，引入上下文压缩机制。当前系统已实现 ContextCompressor 模块(约 900 行代码)，支持 Token 估算和消息摘要，但尚未集成到 REPL 的主流程中。完成集成后，系统将能够在长对话场景下自动压缩历史消息，避免超出模型的上下文窗口限制。

第四，实现 A2A(Agent-to-Agent)协议支持，使不同平台的 Agent 能够相互发现和调用。第五，优化工具调用的安全性，引入沙箱机制限制工具的执行权限，防止恶意操作。第六，实现工作流模板市场，用户可以分享和复用社区贡献的工作流模板。



图 3-1 GrassFlow 系统架构图



图 4-1

自动化测试运行结果

表 4-1 各模块测试用例统计

| 模块       | 测试文件                    | 用例数   |
|----------|-------------------------|-------|
| DAG 引擎   | test_dag.py             | 约 80  |
| 调度器      | test_scheduler.py       | 约 100 |
| DSL 解析器  | test_dsl_parser_v2.py   | 约 120 |
| 工具系统     | test_tools.py           | 约 90  |
| Agent 组件 | test_agent_component.py | 约 100 |
| REPL 终端  | test_repl.py            | 约 80  |
| 其他模块     | 多个测试文件                  | 约 800 |

## 参考文献

- [1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems. 2017: 5998-6008.
- [2] OpenAI. GPT-4 technical report[EB/OL]. (2023-03-27)[2024-01-15]. <https://arxiv.org/abs/2303.08774>.
- [3] Anthropic. Claude 3.5 sonnet model card[EB/OL]. (2024-06-20)[2024-07-01]. <https://docs.anthropic.com/en/docs/about-claude/model-card>.
- [4] DeepSeek-AI. DeepSeek-V3 technical report[EB/OL]. (2024-12-27)[2025-01-10]. <https://arxiv.org/abs/2412.19437>.
- [5] Yao S, Zhao J, Yu D, et al. ReAct: synergizing reasoning and acting in language models[C]//International Conference on Learning Representations. 2023.
- [6] Xi Z, Chen W, Guo X, et al. The rise and potential of large language model based agents: a survey[EB/OL]. (2023-09-14)[2024-03-20]. <https://arxiv.org/abs/2309.07864>.
- [7] Wang L, Ma C, Feng X, et al. A survey on large language model based autonomous agents[J]. Frontiers of Computer Science, 2024, 18(6): 1-26.
- [8] Kahn A B. Topological sorting of large networks[J]. Communications of the ACM, 1962, 5(11): 558-562.
- [9] MongoDB Inc. LiteLLM: a lightweight library to call 100+ LLM APIs[EB/OL]. (2023-10-01)[2025-03-15]. <https://github.com/BerriAI/litellm>.
- [10] Montgomerie A. Rich: a Python library for rich text and beautiful formatting[EB/OL]. (2020-05-01)[2025-03-15]. <https://github.com/Textualize/rich>.
- [11] Slavin R, Hosseini R. prompt\_toolkit: a library for building powerful interactive command lines in Python[EB/OL]. (2014-01-01)[2025-03-15]. <https://github.com/prompt-toolkit/python-prompt-toolkit>.
- [12] Ramshaw L, Marcus M. Text chunking using transformation-based learning[C]//Proceedings of the Third Workshop on Very Large Corpora. 1995: 82-94.
- [13] Python Software Foundation. asyncio — Asynchronous I/O[EB/OL]. (2024-01-01)[2025-03-15]. <https://docs.python.org/3/library/asyncio.html>.
- [14] FastAPI. FastAPI framework, high performance, easy to learn, fast to code, ready for production[EB/OL]. (2024-01-01)[2025-03-15]. <https://fastapi.tiangolo.com/>.
- [15] SchemaOrg. JSON Schema: a vocabulary that allows you to annotate and validate JSON documents[EB/OL]. (2024-01-01)[2025-03-15]. <https://json-schema.org/>.